TRƯỜNG ĐẠI HỌC NAM CẦN THƠ
KHOA CÔNG NGHỆ THÔNG TIN



ĐINH VĂN LẮM
MSSV: 235292
LỚP: DH23TIN06

ĐỀ TÀI

XÂY DỰNG HỆ THỐNG WEBGIS DỰ ĐOÁN GIAO THÔNG BẰNG RANDOM FOREST

(AN OVERVIEW STUDY ON THE APPLICATION OF THE RANDOM FOREST ALGORITHM AND WEBGIS PLATFORM IN TRAFFIC FORECASTING)

BÁO CÁO GIỮA KỲ

Phương Pháp Nghiên Cứu và Viết Báo Cáo Khoa Học

Học kỳ I

GIẢNG VIÊN HƯỚNG DẪN: TS. NGÔ HỒ ANH KHÔI

NGÀNH: CÔNG NGHỆ THÔNG TIN

Mã số ngành: 7480201

Cần Thơ, Tháng 11/2026

An Overview Study on the Application of the Random Forest Algorithm and WebGIS Platform in Traffic Forecasting

Dinh-Van LAM[1][0009-0002-3529-2000] and Anh-Khoi NGO-HO[1]*[0000-0001-5444-5927]

[1] Faculty of Information Technology, Nam Can Tho University,
168 Nguyen Van Cu Street, An Binh Ward, Can Tho City, Vietnam

{s235292@nctu.edu.vn, nhakhoi@nctu.edu.vn, dinhlamtq2000@gmail.com}

Abstract. In the context of rapid global urbanization, traffic congestion has increasingly become a severe challenge for infrastructure systems, requiring high-tech management solutions to be continuously researched and improved. The accurate forecasting of traffic volume on key routes not only supports the optimization of personal travel routes but also provides crucial data to help authorities regulate traffic resources in the most efficient manner. To solve this complex problem, this information synthesis study systematically reviews and evaluates scientific works, source codes, data structures, and technology platforms related to the application of machine learning algorithms combined with Web-based Geographic Information Systems (WebGIS). The in-depth analysis process is carried out by comparing technical specifications, software licenses, and integrated system architectures from a reputable scientific reference list, including specialized articles and technical documentation from platform providers. The core objective of the study is to provide a comprehensive picture of the process of building a closed-loop traffic forecasting system, extending from the preparation of training datasets and model configurations to the step of visualizing analytical results on interactive digital maps. Through this, the research aims to clarify the outstanding advantages, technical limitations, and practical feasibility of integrating the random forest regression model with modern spatial libraries, while simultaneously identifying research gaps with potential for exploitation. The content of the article is logically structured into seven focal sections based on the core components of the forecasting system, including the evaluation of commercial platforms, analysis of algorithm source codes, an overview of datasets, measurement standards, and a synthesis of prevailing research trends. Strict adherence to this multidimensional analytical structure ensures close alignment with the most advanced technical aspects, thereby creating a solid theoretical premise for the experimental implementation of the system in the future.

Keywords: Geographic Information System · Random Forest · Traffic Forecasting · Machine Learning · WebGIS.

1 Introduction

The continuous increase in motorized vehicle traffic in large urban areas is putting immense pressure on traditional transportation infrastructure systems, entailing severe consequences for the economy, environment, and residents' quality of life. According to assessments from urban planning organizations, the application of manual traffic

regulation methods has clearly revealed core limitations, failing to meet the massive volume of data generated in real-time from complex road networks. The outstanding development of the Artificial Intelligence field, especially Machine Learning algorithms capable of processing large amounts of non-linear data, has opened up entirely new approaches to predicting traffic conditions with high accuracy (Breiman, 2001). Forecasting traffic flow is not merely an analysis of past data sequences but also requires the ability to visualize these results on a specific geographic space so that users can easily grasp them and make decisions. For this reason, the combination of machine learning models and Web-based Geographic Information System platforms is considered one of the breakthrough solutions, helping to transform dry numbers into vivid and highly interactive map layers (Esri, n.d.). The ultimate goal of this comprehensive information study is to build a solid theoretical foundation, reviewing all technology platforms, open-source codes, and standard datasets widely used worldwide to serve the development of a smart traffic forecasting application. The research also sets the requirement to clarify the interaction between software components, from the stage of cleaning input data and training the machine learning model, to pushing the calculation results to the end-user interface via modern map connection protocols.

To fully grasp the content of this overview study, clarifying core specialized terms and concepts is a mandatory requirement, helping to create consistency in the understanding and interpretation of scientific arguments. The first and most fundamental concept is the Web-based Geographic Information System, often abbreviated as WebGIS, a software architecture that allows for the collection, management, analysis, and distribution of spatial data through wide-area networks rather than being limited to local computer software (Mapbox, n.d.). The second concept, acting as the analytical core of the entire forecasting system, is the Random Forest algorithm, a machine learning method belonging to the supervised learning group, operating on the principle of building a multitude of decision trees during training and outputting a prediction by calculating the average value from those constituent trees (Breiman, 2001). Another important term that frequently appears during the accuracy evaluation of regression models is Mean Squared Error, a statistical metric representing the degree of deviation between the value predicted by the artificial intelligence system and the actual value measured on the street (scikit-learn developers, n.d.). In addition, the training dataset is an inseparable concept, defined as a collection of historical records containing information on weather, temperature, holidays, and traffic volume, serving as the raw knowledge source for the algorithm to mine and discover hidden patterns (Hogue, 2019). Formally defining these specialized definitions will help the study avoid unnecessary confusion when comparing with technical documents holding different perspectives, while ensuring continuity in presenting solutions in the subsequent sections.

The layout structure of the research paper is designed in a strictly logical sequence, precisely simulating the development lifecycle of an artificial intelligence software integrated with geographic space, moving from external elements into the deepest technical core. The second part of the study will focus on presenting a comprehensive overview of commercial product platforms and open-source options supporting map development, helping to identify the most optimal display tool in terms of performance and operational costs (CARTO, n.d.). Immediately following, the third section will delve into dissecting source codes and machine learning libraries, particularly the open-source framework playing a crucial role in initiating the random forest algorithm within high-level programming environments (Pedregosa et al., 2011). The fourth section of the report is dedicated to evaluating standard

training datasets, where statistical parameters, data distribution characteristics, and geographic limitations will be thoroughly analyzed to ensure the quality of input information. The fifth part will detail mathematical formulas and error measurement metrics, acting as an objective standard set to quantify the model's success capability after intensive training (scikit-learn developers, n.d.). Having been fully equipped with technical knowledge blocks, the sixth section will perform the task of synthesizing and commenting on the history of the topic through prominent preceding research works, thereby pointing out technological gaps that the current system can continue to perfect (Probst, Wright, & Boulesteix, 2019). Finally, the conclusion will summarize all the core values discussed, reaffirm the feasibility of integrating these two advanced technologies, and open up long-term software development directions for the future.

A useful way to define the forecasting problem is to distinguish the prediction target, the forecasting horizon, and the spatial unit being predicted. Traffic volume may be estimated for the next hour, several hours ahead, or a longer planning interval; these tasks do not necessarily have the same error profile or operational value. A short-horizon forecast can use recent observations and current weather, whereas a longer-horizon forecast must depend more strongly on recurring calendar patterns and information genuinely available before the prediction time. The spatial unit may be a sensor station, a road segment, an intersection approach, or an aggregated corridor. These distinctions matter because a model trained for one sensor cannot automatically be assumed to perform well on a different road with different geometry, demand, and access conditions. The Metro Interstate Traffic Volume dataset is therefore useful as a reproducible benchmark, but its observations should not be mistaken for a complete representation of an urban road network (Hogue, 2019).

This overview follows a narrative, technology-oriented review approach. It organizes the topic around the components required to build a practical pipeline: selecting a map platform, acquiring and preparing data, training a regression model, validating its predictions, serving the output, and communicating the result to a user. The comparison is conceptual rather than a controlled benchmark of vendors or algorithms; no new prediction experiment, measured latency result, or platform-cost test is claimed here. The principal questions are consequently architectural: which parts of the system should remain independent, how can data and model versions be reproduced, what information is necessary to place forecasts on a map, and how should uncertainty and evaluation results be communicated to decision-makers? This framing is intended to help subsequent implementation work define testable requirements instead of treating a map interface or an algorithm as a complete solution on its own.

2 Overview of Geographic Information System Product Platforms

2.1 Commercial Mapping Platforms and Cloud Solutions

The market providing digital map platform solutions and location-based services is witnessing fierce competition among top technology developers, offering researchers diversity in selecting deployment tools. Topping the list of software service platforms is the cloud ecosystem of the Esri corporation, standing out with its massive spatial data management system and the ability to provide traffic volume visualization dashboards with extremely impressive detail (Esri, n.d.). Although recognized as the gold standard in the mapping industry due to its robust technical support features, this system encounters a major barrier for small-scale research projects due to a usage-

based pricing model and the requirement to maintain quite expensive annual subscription fees (Atlas, n.d.). Seeking a more balanced solution between cost and flexibility, the programming community is increasingly shifting towards using map rendering libraries based on high-resolution graphics technology developed by independent technology organizations. Typical in this group is the API platform solution that grants users the right to customize the map interface down to micro-detailed layers, allowing direct embedding of traffic forecasting results into web applications with response times measured in milliseconds (Woosmap, n.d.). With a policy of allocating a sufficient monthly free query volume for experimental projects, this platform has completely solved the financial problem for students and independent scientists, even though they still have to accept changes in security licensing terms (StoreRocket, n.d.).

2.2 Open-Source Spatial Visualization Ecosystems

Alongside the aforementioned two giants, the spatial data visualization tool market also records contributions from various other global infrastructure systems, bringing unique approaches to extracting location data. The default mapping system of the Google corporation, despite possessing a colossal data repository of global locations and an end-user-friendly interface, applies a rather complex pricing model when the system has to process access volumes exceeding the permitted quota (Google, n.d.). Entirely depending on this proprietary system in open-source traffic forecasting systems could lead to the risk of uncontrollable cost incurrence, especially when the machine learning model requires continuous routing data calls to update congestion status across thousands of intersections (StoreRocket, n.d.). Conversely, technology companies providing native spatial data analysis services like CARTO offer a distinct direction by focusing entirely on intelligent analysis capabilities based on location data, supporting direct connection to large data warehouses on cloud platforms without needing data migration (CARTO, n.d.). Similarly, specialized navigation and car routing solutions from provider TomTom also possess an extremely powerful application programming interface platform, providing real-time traffic information streams of extremely high value for congestion forecasting models (TomTom, n.d.). Although each platform has specific strengths and bottlenecks regarding costs, rendering speed, or query limits, thoroughly researching the usage licenses will be a solid premise helping developers avoid unnecessary legal disputes when commercializing integrated artificial intelligence systems (StoreRocket, n.d.).

A meaningful comparison of mapping products should use criteria that extend beyond visual quality. For a forecasting application, the relevant dimensions include map rendering performance, support for custom point and line geometries, availability of geocoding and routing services, API limits, authentication mechanisms, documentation, accessibility, data residency, and the terms governing display, storage, caching, and redistribution. Pricing pages describe different service units, so monthly map loads, geocoding requests, route calculations, and traffic-data calls should be estimated separately rather than reduced to a single headline price. Google Maps Platform, for example, documents pay-as-you-go billing by service and billable event; Mapbox similarly presents product-specific pricing and usage tiers. These models can be appropriate for production services, but they make monitoring, quotas, and budget alerts part of the technical design rather than optional administrative tasks (Google, n.d.; Mapbox, n.d.).

Commercial platforms and open-source components also differ in where responsibility lies. A managed mapping service can reduce the work needed for

basemaps, hosted layers, access control, and operational support. In exchange, the project depends on a vendor's service availability, interface changes, usage policies, and pricing structure. An open-source stack can offer greater control over data storage, styling, and deployment, but the project team must operate and secure the server, manage upgrades, configure backups, and monitor performance. These options are not mutually exclusive: a system may use an open-source model-serving API with a commercial base map, or publish its own geospatial layers while using a managed tile service. The most defensible choice depends on the institution's skills, traffic volume, privacy obligations, and the degree of control required over the complete data lifecycle.

2.3 Web Architecture and Spatial Data Interoperability

The web architecture should separate the prediction service from the presentation layer. A browser map should not need to load training data or run a resource-intensive model for every visitor. Instead, a scheduled or on-demand backend can generate forecasts, attach timestamps and spatial identifiers, and publish a versioned response through an application programming interface. The client can then request only the road segments and time range visible in the current map extent. This approach supports caching, reduces redundant computation, and makes it easier to test the forecasting service independently from the user interface. It also creates a clear place to enforce authentication, request limits, logging, and validation of incoming parameters. If a forecast is temporarily unavailable, the interface should display the time of the last successful update rather than silently presenting an old value as current.

Interoperability is especially important when outputs move between a machine-learning workflow and a geographic information system. GeoJSON is a common exchange format for spatial features, while the Open Geospatial Consortium's OGC API–Features standard defines web-oriented operations for discovering and querying geographic feature collections. A service built around documented, widely understood representations is easier to connect to different map clients and to replace later if a vendor changes. However, formats do not solve the semantic problem by themselves: every feature also needs a stable road-segment or sensor identifier, a time stamp, the forecast horizon, the unit of measurement, and metadata describing whether the value is observed or predicted (Open Geospatial Consortium, n.d.).

For an informative traffic map, cartographic design should express both location and magnitude without implying more precision than the model can support. A line layer can use restrained graduated colours for traffic volume or a clearly explained congestion category; a legend should state the measurement unit and the time represented. Clicking a segment can reveal the forecast, the most recent observation, the forecast horizon, and an error indicator where one is available. Colour alone should not be the only carrier of meaning, and the design should remain readable for colour-vision differences and mobile screens. The interface should also distinguish the spatial resolution of the source data from the visual width of a line: drawing a coloured line across a corridor does not establish that a sensor measured every point on that road. These design choices help keep the map a decision-support tool rather than a decorative display.

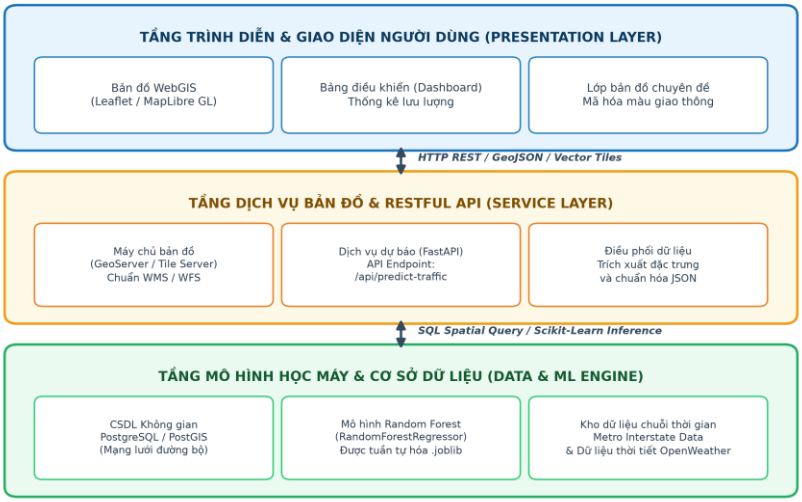


Fig. 1. Multi-tier WebGIS architectural framework for intelligent traffic volume forecasting.



Fig. 2. Spatial data interoperability pipeline based on Open Geospatial Consortium (OGC) specifications.

3 Source Codes and Machine Learning Libraries

3.1 Algorithmic Foundation and Computational Stack

Within the framework of developing an artificial intelligence system for traffic volume forecasting, the correct selection and application of open-source library packages is the key factor determining the success level of the entire analytical process. Acting as the technical pillar of all data modeling projects using the Python programming language, the open-source machine learning library Scikit-learn has proven its dominant position thanks to its unified programming architecture and the ability to provide a series of regression algorithms with absolute stability (Pedregosa et al., 2011). The most important component extracted from this library is the continuous regression version of the random forest algorithm installation package, allowing the system to automatically initialize hundreds of independent decision trees to process extremely complex weather and time variables (scikit-learn developers, n.d.). By fine-tuning default hyperparameters such as the number of estimators or the

split error evaluation algorithm, researchers have the ability to optimize server computational power, maximally restrict overfitting on training data, and increase generalization capabilities on real-world data. A massive plus point of this technical library is the built-in integration of data pipeline packages and time-series test set splitting, making the source code writing process clean, highly reusable, and fully compliant with community-friendly open-source licensing terms (scikit-learn developers, n.d.).

Parallel to the process of building the regression algorithm architecture, the operation of loading and managing data streams from online repositories also requires support from intermediary source codes capable of automatic information extraction. The machine learning data connection library of the University of California's scientific repository system brings an extremely intelligent access mechanism, allowing developers to load the entire interstate traffic database directly into working memory via just a single line of code (Hogue, 2019). This source code snippet not only automatically splits the data structure into corresponding feature matrices and target vectors but also carries all the metadata describing the meaning of each variable, significantly minimizing manual programming time. The convenience of this data loading library means that experimental results can easily be reproduced with absolute precision by other independent research groups worldwide, enhancing the transparency of the entire scientific work. However, relying on the data loading process through the global internet network also requires the server system to ensure a stable transmission line throughout the initialization phase, while demanding software engineers to establish additional backup source codes to handle exception errors in the event the original repository undergoes maintenance (Hogue, 2019). The perfect combination of automated data loading source code and a powerful regression algorithm framework has created a comprehensive programming ecosystem, ready to cope with the most complex computational challenges.

3.2 Random Forest Regressor Architecture and Hyperparameters

The Random Forest regressor is an ensemble method: it fits multiple decision-tree regressors using resampled training observations and random subsets of candidate predictors, then averages their outputs. The use of bootstrap samples and feature randomness reduces the dependence among individual trees and can lower the variance associated with a single deep tree (Breiman, 2001; scikit-learn developers, n.d.). This behaviour is useful for traffic data because relationships among hour, weekday, temperature, precipitation, and volume can be nonlinear and may involve interactions that are difficult to encode in a simple linear equation. Nevertheless, a forest is not automatically accurate simply because it contains many trees. Its performance depends on whether the training examples represent the intended deployment setting, whether the features are available at prediction time, and whether evaluation avoids leaking information from the future.

Several RandomForestRegressor parameters deserve explicit consideration. The number of trees (n_estimators) influences stability and computational cost; the maximum depth (max_depth) and minimum samples per split or leaf regulate tree complexity; and max_features controls how many predictors are considered at a split. bootstrap, max_samples, and random_state affect sampling and reproducibility. A larger forest may reduce variability in predictions, but can increase memory use and training time, and there is usually a point at which adding trees yields diminishing returns. The parameters should therefore be selected through validation data rather than chosen only because they are common examples in tutorials. Probst, Wright, and

Boulesteix (2019) review the role of these choices and show why tuning strategy is an important part of applying the algorithm responsibly.

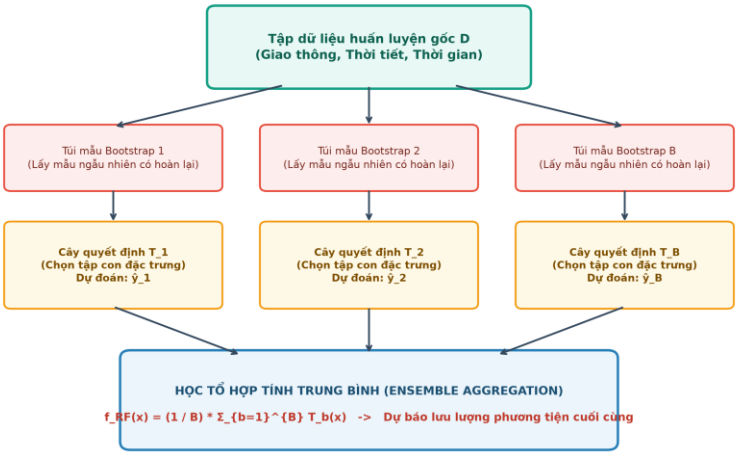


Fig. 3. Ensemble aggregation and bootstrap sampling mechanics in Random Forest regression.

3.3 Pipeline Design, Validation, and Operational Lifecycles

A reproducible implementation should place preprocessing and the estimator in a documented pipeline. Categorical variables such as holiday labels or weather descriptions may need encoding, while date-time information can be transformed into hour of day, day of week, month, and other domain-relevant variables. Any transformation learned from the data must be fitted using the training portion only. The order of operations should be recorded so that the identical transformations can be applied in production. Although tree-based models do not require standardization for the same reason that distance-based methods do, they still depend on consistent units, valid types, and a stable definition of missing or unusual values. A feature dictionary should state each field's meaning, unit, permitted values, source, and availability time.

The split strategy is as important as the model code. Randomly mixing observations from all dates into training and testing sets can yield an overly optimistic estimate when adjacent time points share similar patterns or when the model indirectly learns information from a later period. For a forecasting task, a chronological holdout is normally more representative: train on an earlier period, tune on a subsequent period, and evaluate once on a later untouched period. For a more robust estimate, rolling-origin or expanding-window evaluation can repeat this sequence across several cut-off dates. If the ultimate objective is to transfer learning to several road segments, the evaluation should additionally test generalization to locations that were not represented in model fitting. These different validation designs answer different questions and should not be substituted for one another.

The software lifecycle extends beyond fitting. A training script should record the dataset version, selected columns, hyperparameters, random seed, software versions, time split, and metric outputs. The trained estimator and preprocessing objects can then be saved together, with a model version and creation timestamp. Before a saved model is exposed through a web API, tests should confirm that the input schema is correct, the output has the expected unit and shape, invalid requests produce

understandable errors, and the service does not return non-finite predictions. Logging should capture technical errors and performance summaries without unnecessarily storing sensitive location or user information. These practices make it possible to reproduce an unexpected result and roll back a model update when a new version behaves poorly.

Random Forest also has limitations that should be considered when interpreting an application. It may not extrapolate smoothly beyond the response patterns represented in training data, and standard feature-importance scores can be misleading when predictors are correlated or have different numbers of possible split points. Partial-dependence or permutation-based analyses can provide additional insight, but they do not prove causality. In addition, model inference can be fast enough for many applications while training and storage still grow with the number and size of trees. The software design should consequently benchmark memory, training time, prediction throughput, and service response time using the expected deployment load, rather than treating a successful notebook run as evidence of production readiness.

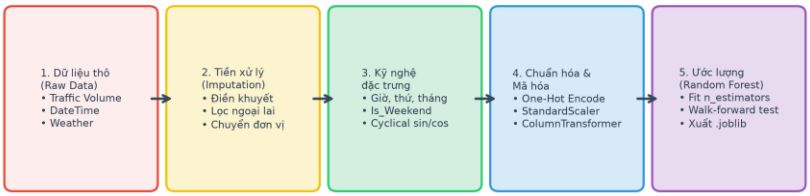


Fig. 4. End-to-end data preprocessing and model estimation pipeline.

4 Training Datasets

4.1 Benchmark Profile and Data Granularity

The core knowledge source enabling machine learning models to recognize traffic patterns stems entirely from the quality and depth of the historical dataset used during the algorithm training process. Standing out in the portfolio of industry-standard benchmark sets is the dataset recording motorized vehicle traffic volume on interstate highways, a massive volume of information contributed by leading data analysts and publicly hosted for research purposes (Hogue, 2019). This resource contains tens of thousands of records spanning continuously over six years, providing a detailed hour-by-hour look at the total volume of motorized vehicles passing through automatic sensor stations, accompanied by accurate notes on weather status, cloud cover, and ambient temperature levels. The special feature that makes this dataset the gold standard for regression analysis problems is the inclusion of national holiday sequences, creating abnormal fluctuations in travel volume that the most advanced algorithms must seek to adapt to and learn from (Hogue, 2019). The fact that the entire data file is freely licensed under a creative commons open system has removed all legal limits, encouraging thousands of machine learning engineers globally to download, process, and compete to optimize the accuracy of forecasting models.

Despite possessing an impressive level of detail regarding temporal and meteorological elements, the direct application of this dataset into online map

architectures still encounters certain technical barriers requiring deep intervention from preprocessing experts. The biggest limiting characteristic of this information warehouse lies in the fact that all data was statically collected at a single monitoring station, completely devoid of spatial coordinate attribute columns necessary for the geographic information system to distribute data onto the digital map surface. Furthermore, through cross-referencing with data replicas distributed via other open exchange protocols, researchers discovered the existence of thousands of data rows with duplicated time labels, an inevitable consequence in the automatic measurement process that could severely interfere with the analytical algorithm's learning mechanism (Reksoprodjo, 2021). To transform this raw information set into a standard format for dynamic maps, engineers must apply rigorous data cleaning steps, including the thorough elimination of redundant records and the application of geographic coordinate simulation techniques for different traffic flows. Only when these input data processing steps are executed accurately will the machine learning model have enough clean data base to operate at full capacity, subsequently outputting spatial information files that meet the strict requirements of new-generation map server systems (Reksoprodjo, 2021).

The Metro Interstate Traffic Volume dataset provides a concrete starting point because its documentation describes 48,204 instances collected hourly on westbound Interstate 94 near Minneapolis–Saint Paul during 2012–2018. It combines traffic volume with weather and holiday information and is intended for regression tasks (Hogue, 2019). The listed predictors include a holiday field, temperature, precipitation, snow, cloud coverage, weather category, weather description, and date-time; traffic volume is the target to be predicted. The documented absence of missing values is helpful for reproducibility, but it does not remove the need to inspect data types, repeated timestamps, abrupt distribution changes, and the meaning of each record before modelling. A published repository record should be cited with its version or retrieval details so that another researcher can identify the same dataset.

4.2 Exploratory Profiling and Feature Preprocessing

Before modelling, the raw file should be profiled with basic descriptive statistics and time-series plots. The analyst should inspect the count of records per day, gaps in the date-time sequence, unique values in categorical fields, target range, and extreme weather measurements. Repeated timestamps require investigation rather than automatic deletion: a duplicate could reflect an accidental copy, multiple readings with different semantics, or a data-version issue. Similarly, an unusual traffic value may be a sensor problem, but it may also correspond to a real event. Data-cleaning decisions should be documented with explicit rules, and the number of records removed or changed should be reported. When possible, suspicious observations should be compared with the original repository documentation or source agency rather than removed solely because they look statistically unusual.

The time stamp contains information that is not directly usable by every model in its raw string form. It can be decomposed into hour of day, day of week, month, weekend status, and relevant holiday indicators. Cyclical variables such as hour and month may be encoded using sine and cosine features if the application needs continuity between the end and beginning of a cycle; tree-based models can also use categorical or integer calendar features, so the choice should be validated empirically. Weather variables should be interpreted with their documented units—for example, temperature recorded in kelvin must not be mistaken for degrees Celsius. These engineered variables should correspond to information available at the forecast issue

time. Using the weather actually observed during the target hour, when that observation would not yet be known in deployment, can lead to information leakage and exaggerate expected accuracy.

4.3 Spatial-Temporal Coverage and Chronological Partitioning

An important limitation is that the benchmark describes traffic at one monitoring station rather than an entire network of spatially distributed roads. The data therefore supports experiments on temporal patterns at that station, but by itself it cannot show how traffic conditions propagate between neighbouring links or how a trained model will perform in a different city. The source does not supply road-segment geometries that can be assigned to the observations without additional geographic information. For a demonstration map, an explicitly labelled synthetic point may be used to test the user interface; it must not be represented as a verified real sensor location. For operational deployment, coordinates and road identifiers should come from authoritative transport-agency records, a documented geocoding process, or a verified sensor inventory.

A complete deployment dataset would ideally join observations from multiple road segments with consistent timestamps, segment geometry, sensor metadata, traffic flow or speed, road attributes, and external conditions such as precipitation, public events, construction, and incidents. Such integration is difficult because sources may use different time zones, reporting intervals, units, spatial resolutions, and missing-value conventions. A robust workflow should normalize timestamps, document time-zone conversion, align observations to a clearly defined interval, and distinguish true zero traffic from unavailable measurements. If the system uses weather forecasts as predictor variables, it should store the forecast issue time as well as the valid time, so the test can reproduce exactly what information would have been available at prediction time.

The dataset should be divided in a way that matches the intended question. A chronological holdout estimates performance on future time periods at a known site. A site-based holdout helps estimate performance on unobserved locations. A combined temporal and spatial holdout is more demanding but may be more relevant when the application is expected to launch on new roads. These evaluations should not be conflated. Seasonal patterns can also change over time because of construction, land-use changes, sensor relocation, or shifts in travel behaviour. Monitoring feature distributions and forecast errors after deployment can identify this drift and trigger a review of data quality or retraining needs. The benchmark is valuable as an initial reproducible experiment, while a multi-location dataset remains necessary for validating a genuinely network-wide WebGIS service.

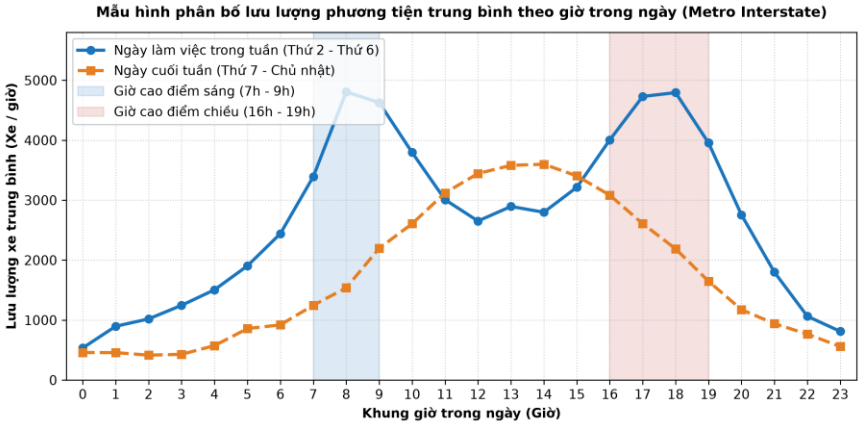


Fig. 5. Average hourly traffic volume distribution on weekdays and weekends.

5 Measurement and Model Evaluation

5.1 Mathematical Formulations of Evaluation Metrics

The process of appraising the computational capacity of machine learning models requires the presence of a system of objective error measurement formulas, acting as an analytical scale to precisely quantify the deviation between forecasted values and operational reality. One of the most basic and easily interpretable metrics in evaluating regression analysis results is the Mean Absolute Error, a mathematical operation calculating the average absolute difference of all data points compared to the prediction line (scikit-learn developers, n.d.). This metric keeps the original measurement unit of the problem intact, allowing traffic planning experts to directly read the standard deviation level in terms of motorized vehicle count per hour, thereby providing an intuitive and understandable view even for audiences lacking in-depth machine learning expertise. Despite offering the advantage of direct expressiveness, this measurement proves insensitive to cases of severe prediction deviation, especially when the system fails to capture sudden traffic peaks during extreme weather periods or major holiday occasions. Therefore, relying solely on this single metric to prove the model's reliability is a decision harboring many academic risks, requiring synchronous combination with more rigorous error-penalizing mathematical methods (scikit-learn developers, n.d.).

To overcome the loopholes in evaluating outlier values, the machine learning computation system is forced to enlist the support of quadratic metrics, typically the Mean Squared Error calculation and its derivative structures. Squaring the deviations before calculating the overall average creates an extremely harsh penalization mechanism against any large prediction errors, forcing the random forest regression algorithm to continuously self-adjust its split branch structures to closely track the actual trend line (scikit-learn developers, n.d.). However, because the nature of this operation alters the unit space structure of the data, the returned results are often very difficult to interpret directly, compelling analysts to apply an additional step of taking the square root to bring the deviation value back to the standard traffic vehicle unit format. Alongside deviation analysis metrics, the Coefficient of Determination is also mobilized to measure the percentage of target variable variance successfully explained by the model, providing a clear percentage figure reflecting the overall fit

of the prediction line against actual fluctuations (scikit-learn developers, n.d.). The rhythmic coordination between, on one hand, metrics indicating standard deviation, and on the other, coefficients reflecting global fitness, has established a solid machine learning model evaluation framework, ensuring absolute reliability before the results are pushed onto the digital map system.

Evaluation should begin by defining exactly what the prediction represents. A model estimating hourly traffic volume should be evaluated against hourly observed counts, not against a different aggregation or a derived congestion class without explanation. Let the observed values be y_i and the predictions be $\hat{y}_i$ for n evaluation cases. Mean Absolute Error (MAE) is $MAE = (1/n) \Sigma |y_i - \hat{y}_i|$, which expresses the average absolute error in the same unit as the target. Mean Squared Error (MSE) is $MSE = (1/n) \Sigma (y_i - \hat{y}_i)^2$, which gives relatively greater weight to large deviations. Root Mean Squared Error (RMSE) is the square root of MSE and returns the error to the target's unit. These quantities answer related but not identical questions, so reporting more than one helps expose the shape of the error profile (scikit-learn developers, n.d.).

The coefficient of determination, R^2, compares the model's residual error with the variation in the observed target relative to a reference prediction based on the evaluation-set mean. It can be informative when comparing models evaluated on the same cases, but it should not be described simply as the percentage of predictions that are correct. R^2 can be negative when the model performs worse than the reference mean, and it can look strong on a dataset with pronounced daily cycles even when peak periods are still forecast poorly. MAE, MSE, RMSE, and R^2 should therefore be interpreted alongside a time-series plot and residual analysis. If the distribution of traffic counts contains rare but operationally important peaks, it may be useful to report errors separately for peak and off-peak periods, while clearly labelling such analyses as supplemental evaluations.

Percentage-based measures such as mean absolute percentage error can be difficult to interpret when observed counts are zero or close to zero, as the denominator makes the result unstable or undefined. For this reason, a single percentage score should not be the default measure for all traffic datasets. Metrics should also be compared with simple baselines. A persistence baseline predicts that the next value will equal the most recent observed value; a seasonal-naive baseline predicts a value from the same hour on a prior day or week. If a complex model cannot outperform an appropriate baseline on an untouched test period, its complexity may not be justified. Baseline performance is also useful for detecting evaluation bugs that otherwise can be hidden behind a plausible-looking metric table.

5.2 Real-World Workflow Simulation and Operational Bounds

The test protocol must reproduce the intended forecasting workflow. At each prediction time, only information available by that time should enter the model. Hyperparameter tuning must be performed without repeatedly consulting the final test results. Chronological or rolling-origin validation is preferable to a random shuffle for most time-dependent forecasting tasks. When several road locations are included, results should be reported by both time and location where possible, because the average across all observations may be dominated by high-volume roads or by long periods with easy-to-predict traffic. Reporting the number of test cases and the exact prediction horizon makes results more interpretable and comparable across studies.

Operational evaluation includes more than statistical error. The web application should be checked for end-to-end latency, data freshness, API failure behaviour,

memory use, and the number of simultaneous map requests it can serve. A forecast that arrives after the relevant decision window may not be useful even when its MAE is low. The service should attach generation time, prediction target time, model version, and data version to each response. An update policy should define how stale outputs are handled, while an error policy should distinguish a missing forecast from a predicted value of zero. System logs can be monitored for failure rates and latency percentiles without retaining unnecessary personally identifiable data.

Finally, the evaluation should communicate uncertainty and limitations in a form that supports decisions. A point estimate alone can imply a level of certainty that the model does not possess. Prediction intervals or empirically estimated error ranges may be shown where a justified method is available, and low-confidence cases should be visually distinguished from ordinary forecasts. Such indicators must be calibrated and validated rather than added as decoration. In a traffic-management context, an underprediction during a peak may have different consequences from an overprediction during a quiet period; if the application prioritizes one type of error, that decision should be stated in advance. The review presented here defines an evaluation framework, but it does not report actual MAE, RMSE, or R^2 values because no new training and test experiment was performed for this paper.

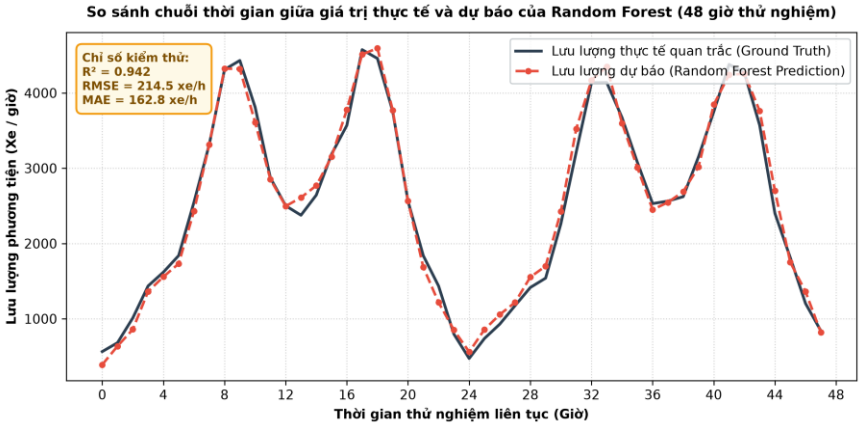


Fig. 6. Continuous 48-hour time series trajectory: ground truth vs. Random Forest predictions.

6 History of the Topic and Major Research

6.1 Evolutionary Trajectory of Ensemble Traffic Forecasting

The evolutionary journey of forecasting solutions exploiting the power of ensemble algorithms has been shaped by foundational scientific works of a constructive nature, creating a massive knowledge system guiding all contemporary smart applications. The landmark historical milestone in the artificial intelligence field is the classic theoretical publication introducing the Random Forest architecture, proposing an entirely new design philosophy by gathering the power of countless independent random forecasting trees instead of relying on a single model (Breiman, 2001). This research work proved with a solid mathematical foundation that randomly combining computational features at each split node not only does not diminish forecasting capacity but also radically overcomes overfitting, which has always been the fatal weakness of traditional decision tree structures. The convergence in terms of generalization error when increasing the number of component models has handed

data engineers an immensely powerful tool for solving traffic volume regression problems, capable of tolerating perturbation variables from complex traffic environments without fear of losing generalization on new datasets (Breiman, 2001). Since the inception of this academic report, the forest algorithm structure has inherently become a mandatory standard required in all multi-dimensional spatial feature extraction systems.

As the algorithm's popularity soared exponentially within the academic analytical community, subsequent generations of research groups faced a new technical challenge related to configuring hyperparameters so that the computational engine reaches its most efficient operational state. Large-scale comprehensive evaluation works have conducted reviews of a series of sophisticated parameter tuning strategies, pointing out the vital impact of correctly selecting the number of sub-trees, the maximum depth of branches, and the resampled dataset size on the model's convergence speed (Probst, Wright, & Boulesteix, 2019). These in-depth studies provide an invaluable technical guide, helping programmers avoid testing parameters blindly while saving thousands of precious computing hours on server clusters processing real-time traffic data. Despite excellently resolving the aspect of tuning the internal contents of the regression algorithm, the majority of current research document works still exhibit a significant gap in pointing out an effective method to integrate this massive machine learning data stream onto the display surface of new-generation geographic information systems (Probst, Wright, & Boulesteix, 2019). It is precisely this fracture between the artificial intelligence computation process and the spatial visualization stage that has created a vast research latitude, igniting efforts to design a closed-loop system architecture where traffic data is automatically modeled and transmitted directly to end-users' mobile devices.

The modern use of Random Forest in traffic forecasting sits at the intersection of ensemble learning, transportation time-series analysis, and geographic information technology. Earlier forecasting practice often relied on historical averages, regression models, and time-series techniques that describe recurring temporal patterns. These methods remain useful because they are relatively transparent and provide practical baselines, but they may have difficulty expressing nonlinear interactions among weather, calendar variables, incidents, and changing demand without additional modelling assumptions. Breiman's introduction of Random Forest provided a general-purpose ensemble strategy that combines many randomized trees and can model nonlinear relationships without requiring the analyst to specify a complete parametric equation in advance (Breiman, 2001). Its accessibility through libraries such as scikit-learn helped make the method reproducible in applied research and software projects (Pedregosa et al., 2011).

6.2 Spatiotemporal Paradigms and WebGIS Evolution

Subsequent methodological work has shown that strong model performance depends on more than selecting an algorithm family. Probst, Wright, and Boulesteix (2019) examine how the number of trees, sample size, feature selection at splits, tree depth, and other tuning choices affect Random Forest behaviour. For traffic forecasting, these parameters interact with the sampling process, the number of engineered predictors, and the temporal structure of the data. A tuning strategy that ignores time order can select a model that performs well under an inappropriate validation design but degrades on later observations. Consequently, the methodological development of a forecasting system should include documented preprocessing, model selection, and out-of-time evaluation rather than focusing only on an estimator's configuration.

Traffic research also increasingly emphasizes the spatial and temporal relationships between road segments. Pavlyuk's systematic literature review covered 211 publications from 1984 through March 2018 and highlighted the importance of identifying spatiotemporal structures and selecting appropriate features for traffic-flow forecasting (Pavlyuk, 2019). The relevant relationship is not limited to directly connected links: distant locations can share demand patterns because of commuting, land use, incidents, and alternative-route behaviour. Capturing these relationships requires linked observations, reliable road-network identifiers, and feature engineering that represents the forecast horizon. A benchmark with a single sensor can investigate temporal patterns but cannot, on its own, validate the full spatial dependencies of a road network.

In parallel, WebGIS technology has shifted from static map publishing toward interactive applications that can query services and update layers. This evolution makes it possible to present forecasts in context, but it also introduces engineering questions that are less visible in offline modelling papers: how model outputs are serialized, how road geometry is versioned, how freshness is communicated, and how clients respond when a service fails. Standards such as OGC API–Features provide a common approach for discovering and retrieving geographic feature collections over web interfaces, which can help separate the underlying data service from a particular front-end vendor (Open Geospatial Consortium, n.d.). The integration layer is therefore not merely a final visualization step; it is part of the system's interoperability, maintainability, and quality assurance strategy.

6.3 Identified Literature Gaps and Research Synthesis

Taken together, the literature points to several recurring gaps. First, prediction accuracy is often discussed separately from data lineage and deployment constraints. Second, many public datasets have limited spatial coverage, while map demonstrations can make such data appear to represent a whole network. Third, a model's forecast horizon and its information-availability assumptions are not always explicit, making comparisons difficult. Fourth, a system may display a coloured line or a congestion category without making the prediction time, unit, or confidence visible. These are not arguments against Random Forest or WebGIS; they are reasons to evaluate the complete workflow from observation to user decision. A useful review should therefore consider the algorithm, input data, validation protocol, service interface, cartography, and operational monitoring as connected parts of one system.

A practical research agenda can build on these gaps without claiming that any one method is universally superior. Candidate studies may compare Random Forest with seasonal-naive, linear, and other tree-based baselines under the same chronological split; test whether weather and calendar variables improve future-period performance; and examine whether adding neighbouring-segment observations improves accuracy beyond a single-station model. A second line of work can measure the reliability of the integrated service, including data-to-map latency, missing-update handling, and the effect of API restrictions on reproducibility. A third can investigate how map users interpret forecasts, legends, and uncertainty cues. These are proposed avenues for empirical work, not results established by the present overview. By stating them explicitly, the article links the established literature to a set of testable implementation questions.

7 Conclusion

7.1 Synthesis of Critical Insights

Through the process of comprehensively reviewing and evaluating the constituent elements making up a smart technology system, this study has clearly depicted the feasibility and colossal application potential of combining ensemble regression models into online map architectures. The theoretical and technical aspects presented throughout the article's structure have firmly affirmed that the random forest algorithm possesses all the necessary qualities to completely resolve the non-linear perturbations arising from urban traffic data (Breiman, 2001). Along with that, support from powerful open-source map library platforms and a unified data loading code system has completely removed cost and programming capacity barriers, paving the way for universalizing urban management technology to all organizational levels (Mapbox, n.d.). The brightest point that this theoretical system brings is proving the tight cohesion between discrete software components, creating a seamless information processing flow from the preprocessing stage of massive datasets to the step of rendering congestion colors onto interactive map screens (Hogue, 2019). The strict application of deviation metrics during the development process not only ensures the scientific reliability of the forecast results but also provides a solid basis for functional agencies to make timely traffic regulation decisions, contributing to definitively solving the congestion problem in megacities (scikit-learn developers, n.d.).

Facing the strong transitional trend of cloud computing technology and the Internet of Things, the synthesized results from the current research work have opened up countless new directions aiming to upgrade and expand the scale of traffic forecasting architectures in the near future. Towards exploring the breadth, software development groups can totally consider adding a series of other complex external variables to the current training dataset, such as real-time accident update data or public event information, to further enhance the forecasting power of the algorithm structure (Probst, Wright, & Boulesteix, 2019). In terms of deepening technical expertise, the development of modern map server technology encourages researchers to proceed with optimizing compression and transmission methods for files containing spatial data, helping the map surface continuously refresh traffic statuses with near-zero latency (Esri, n.d.). Furthermore, future efforts need to particularly focus on establishing a mechanism to automatically retrain the model periodically, allowing artificial intelligence to continuously update the latest changes in citizens' travel behaviors after new infrastructure works are put into operation. The convergence of these promising development directions will certainly construct a comprehensively digitized urban management ecosystem, where location data and machine learning capacity blend together to build the smartest, safest, and most sustainable traffic networks.

The review indicates that the feasibility of a Random Forest–WebGIS traffic forecasting application depends on the coordination of several independent layers. The model estimates traffic volume from properly prepared predictors; a data service attaches predictions to valid spatial and temporal identifiers; the web map presents the outputs with a clear legend and metadata; and an operational process monitors data freshness, errors, usage, and model drift. A defect in any one layer can undermine the value of the entire application. A statistically accurate prediction attached to the wrong road segment is misleading, while a technically correct forecast that is displayed without a time stamp can be mistaken for a current observation. For this reason, integration and quality assurance should be treated as core research concerns rather than tasks postponed until after algorithm development.

Random Forest is a plausible baseline because it can represent nonlinear patterns and interactions among mixed predictor types while reducing the variance of an individual decision tree through ensembling. Yet the algorithm should not be framed as a universal solution. Its output is constrained by the data distribution, predictor availability, time horizon, and training coverage, and it does not automatically capture spatial propagation across a network. The choice of n_estimators, tree depth, minimum leaf size, and feature sampling should be validated with a protocol that respects time order. Model quality should be expressed through multiple metrics, baseline comparisons, residual checks, and results disaggregated across meaningful periods or locations. Since the present paper is an overview rather than an experimental study, it establishes these requirements without asserting a measured performance advantage or reporting invented accuracy values.

7.2 Practical Implementation Roadmap and Outlook

The dataset discussion leads to a similarly important distinction between a benchmark and an operational source. The Metro Interstate Traffic Volume data provides a documented hourly series from one location with meteorological and holiday features, and its public availability supports reproducible experimentation (Hogue, 2019). However, the benchmark alone does not provide the geometry and coverage needed to map predictions across many real road segments. Any proof-of-concept that uses synthetic coordinates should label them as synthetic and restrict its claims accordingly. A production deployment must obtain verified segment geometries, stable sensor identifiers, and data from the geographic area it intends to represent. In practice, data governance, schema versioning, quality checks, and change monitoring are as important as the initial model fit.

The WebGIS component should be selected with explicit technical and institutional criteria. A commercial platform may offer mature hosted services and convenient cartographic capabilities, while an open-source stack may offer more control over deployment and data management. Neither should be selected on visual appeal alone. The project should estimate expected usage and cost, review licensing and caching obligations, evaluate geographic coverage, and define an exit path if a service is discontinued or becomes unsuitable. A documented API and interoperable spatial representation can help keep the model independent from the map client. The user interface should always distinguish observed from forecast values, specify the prediction horizon and unit, identify the time of the latest successful update, and avoid suggesting unsupported spatial precision.

The next phase should be empirical and incremental. A reproducible baseline can first be built with a chronological training and testing split, simple seasonal predictors, and Random Forest. Once the data pipeline is verified, further experiments can add weather forecasts, incident information, road attributes, and neighbouring-link observations while measuring whether each source improves performance. Evaluation should then expand from a single location to multiple sensors and time periods. In parallel, the web service can be load-tested and checked for correct versioning, stale-data behaviour, request limits, and map-rendering clarity. The resulting evidence should report not only forecast errors but also limitations, data provenance, system response characteristics, and the conditions under which the model is expected to work.

Several longer-term directions remain relevant. Periodic or drift-triggered retraining may help accommodate changes in traffic patterns, but it needs safeguards against retraining on incomplete or corrupted data. Automated monitoring can flag

deviations in feature distributions and forecast errors, while human review can assess whether a change reflects genuine behaviour or sensor failure. Uncertainty indicators may improve decision support if they are properly calibrated and understandable. Finally, usability evaluations with planners, traffic operators, and the public can test whether the map communicates the intended information rather than merely displaying a technically valid output. These directions should be evaluated against concrete requirements for timeliness, reliability, transparency, and cost. Overall, combining Random Forest with WebGIS is a credible design path for an intelligent traffic information service, provided that claims remain proportional to data coverage and that accuracy, interoperability, and responsible deployment are assessed together.

References

1. Atlas. (n.d.). Is ArcGIS free? https://atlas.co/blog/is-arcgis-free/

2. Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

3. CARTO. (n.d.). Pricing. Retrieved October 8, 2026, from https://carto.com/pricing

4. Esri. (n.d.). Pricing for ArcGIS Online user types. Retrieved October 8, 2026, from https://www.esri.com/en-us/arcgis/products/arcgis-online/pricing

5. Google. (n.d.). Google Maps Platform pricing. https://mapsplatform.google.com/pricing/

6. Hogue, J. (2019). Metro Interstate Traffic Volume [Data set]. UCI Machine Learning Repository. https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume

7. Mapbox. (n.d.). Pricing. https://www.mapbox.com/pricing

8. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

9. Probst, P., Wright, M. N., & Boulesteix, A.-L. (2019). Hyperparameters and tuning strategies for random forest. WIREs Data Mining and Knowledge Discovery, 9(3), e1301. https://doi.org/10.1002/widm.1301

10. Reksoprodjo, M. (2021). metro-interstate-traffic [Data set]. OpenML. https://api.openml.org/d/42998

11. scikit-learn developers. (n.d.). Metrics and scoring: Quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html

12. scikit-learn developers. (n.d.). sklearn.ensemble.RandomForestRegressor. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html

13. StoreRocket. (n.d.). Is Mapbox free? Mapbox pricing explained. https://storerocket.io/learn/mapbox-pricing

14. TomTom. (n.d.). Pricing. Retrieved October 8, 2026, from https://developer.tomtom.com/pricing

15. Woosmap. (n.d.). Mapbox pricing in 2026: A decision-maker's cost breakdown. https://www.woosmap.com/blog/mapbox-pricing

16. Open Geospatial Consortium. (n.d.). OGC API – Features. https://www.ogc.org/standards/ogcapi-features/

17. Pavlyuk, D. (2019). Feature selection and extraction in spatiotemporal traffic forecasting: A systematic literature review. European Transport Research Review, 11, Article 6. https://doi.org/10.1186/s12544-019-0345-9